

SLIIT

Faculty of Computing

SE3020 — Distributed Systems

BSc (Hons) in Information Technology — Specialised in Software Engineering

Assignment 1 — Project Report

Clinexa

An AI-Enabled Smart Healthcare Appointment and Telemedicine Platform Built on
Microservices Architecture

Full Name	Registration Number	Role
Joel Nithushan A.T	IT23556652	Team Leader
Vaishnavi L	IT23717336	Member
Thushalini U	IT23794870	Member
Kanistan T	IT23748644	Member

Group ID: NU_04

Date: 17 April 2026

Table of Contents

1. Introduction	3
2. System Architecture	3
2.1 Architectural Overview and Diagram	3
2.2 Microservices Summary	4
2.3 Technology Stack	4
2.4 Database Mapping	5
3. Service Interfaces	5
3.1 Authentication Service	5
3.2 User Service	6
3.3 Doctor Service	6
3.4 Prescription Service	7
3.5 Appointment Service	7
3.6 Payment Service	7
3.7 Notification Service	8
3.8 Telemedicine Service	8
3.9 Symptom Checker Service	8
3.10 Admin Endpoints	9
4. Key Workflows	9
5. Security and Authentication Mechanisms	12
6. Deployment Guide	13
7. Individual Contributions	14
8. Conclusion	14
Appendix A Code Listings	15

1. Introduction

The healthcare industry has been undergoing a significant transformation driven by digital technologies, particularly in the areas of remote consultations, electronic health record management, and AI-assisted diagnostics. Clinexa is a cloud-native healthcare platform developed in response to these evolving needs, drawing inspiration from real-world telemedicine systems such as Channeling.lk, oDoc, and mHealth. The platform enables patients to discover qualified doctors, book appointments, attend secure video consultations, upload medical documents, and receive preliminary AI-generated health suggestions.

The system is architected using the Microservices pattern, where each functional domain is implemented as an independently deployable service. This architectural choice ensures that individual components can be developed, scaled, and maintained without affecting the rest of the system. The platform exposes a unified RESTful API through a Spring Cloud Gateway, which handles cross-cutting concerns such as JWT-based authentication and request routing.

From an infrastructure perspective, the entire platform is containerised using Docker and orchestrated through Kubernetes, making it straightforward to deploy consistently across environments. The frontend is a React.js single-page application served through Nginx. Third-party integrations include Stripe for payments, Twilio for SMS, JavaMail for email, Jitsi Meet for video, and Google Gemini for AI-powered symptom analysis.

2. System Architecture

2.1 Architectural Overview and Diagram

Clinexa is built on a distributed microservices architecture. All client requests originate from the React.js frontend and pass through the Spring Cloud Gateway, which validates JWT tokens, applies routing rules, and forwards requests to the correct downstream service. Each microservice maintains its own isolated PostgreSQL database, adhering to the database-per-service pattern central to a loosely coupled microservices design.

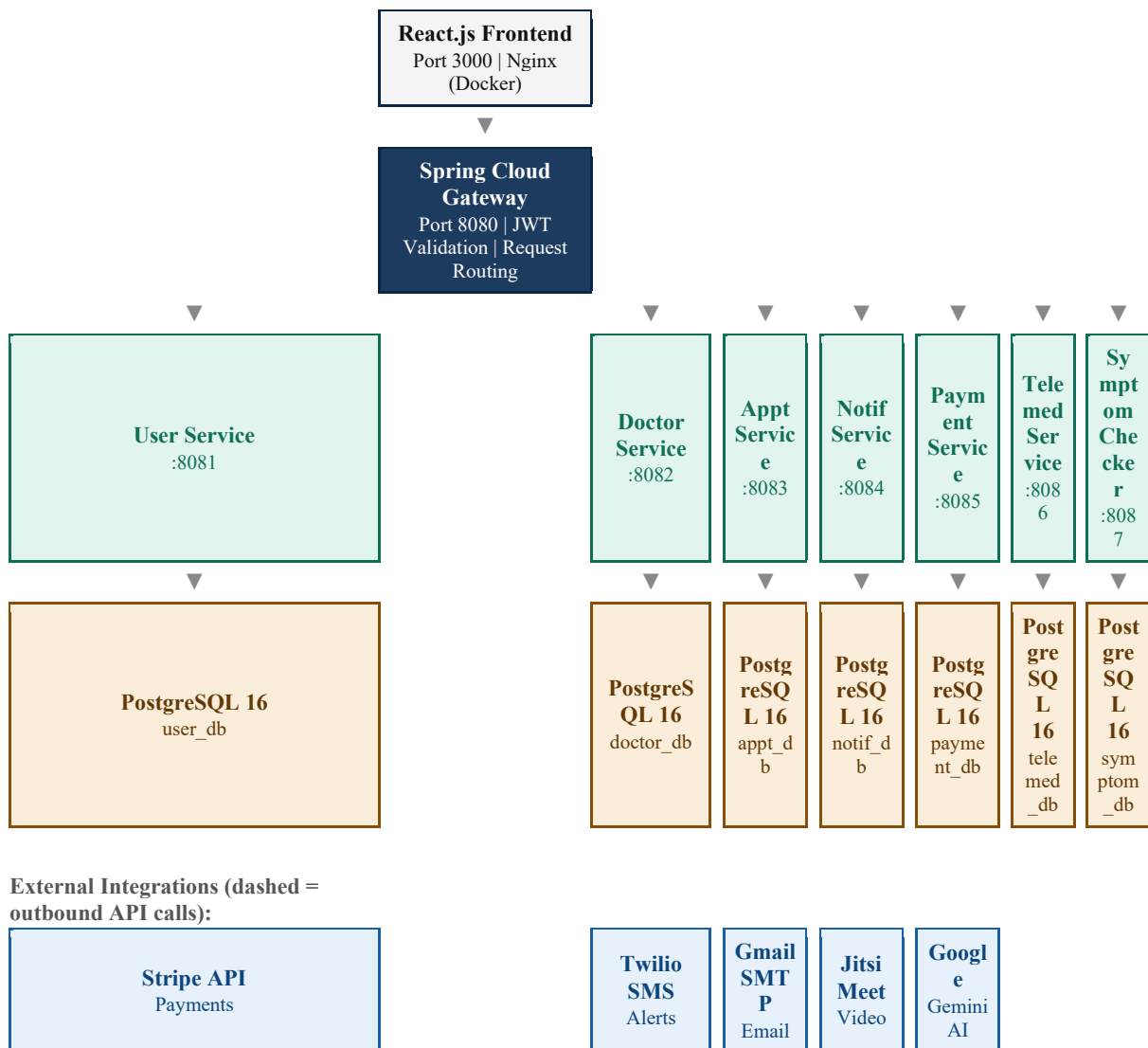


Figure 1 — Clinexa Platform High-Level Architecture Diagram

2.2 Microservices Summary

The platform comprises eight microservices, each designed around a specific functional area. The table below provides a brief description of each service and its assigned port.

Service	Port	Responsibility
API Gateway	8080	Routes all requests, enforces JWT authentication, and handles load balancing
User Service	8081	Manages patient and admin accounts, profiles, and medical report uploads
Doctor Service	8082	Manages doctor profiles, specialisations, availability, and verification status
Appointment Service	8083	Handles appointment booking, modification, cancellation, and status tracking
Notification Service	8084	Sends SMS alerts via Twilio and email notifications via JavaMail over Gmail SMTP
Payment Service	8085	Processes consultation payments through the Stripe Payment Intents API
Telemedicine Service	8086	Creates and manages video consultation sessions via Jitsi Meet
Symptom Checker Service	8087	Accepts patient-reported symptoms and queries Google Gemini for AI-based analysis

2.3 Technology Stack

Layer	Technology
Frontend	React.js (Create React App), served via Nginx inside Docker
Backend	Spring Boot (Java 17) — all eight microservices
API Gateway	Spring Cloud Gateway (reactive, Netty-based)
Database	PostgreSQL 16 (containerised, one database per service)
Security	JWT (JSON Web Tokens) with BCrypt password hashing
Video Consultation	Jitsi Meet (open-source, public deployment)
Payment Processing	Stripe Payment Intents API (sandbox environment)
SMS Notifications	Twilio Programmable SMS
Email Notifications	JavaMail via Gmail SMTP
AI Symptom Analysis	Google Gemini API
Containerisation	Docker with multi-stage builds
Orchestration	Kubernetes (Minikube/Kind) and Docker Compose

2.4 Database Mapping

Each microservice maintains its own isolated PostgreSQL database to ensure loose coupling and independent scalability. All databases run within Docker containers and are mapped using the Docker Compose configuration.

Service	Database Name	Service Port
User Service	user_db	8081
Doctor Service	doctor_db	8082

Service	Database Name	Service Port
Appointment Service	appointment_db	8083
Notification Service	notification_db	8084
Payment Service	payment_db	8085
Telemedicine Service	telemedicine_db	8086
Symptom Checker Service	symptom_checker_db	8087
API Gateway	— (no database)	8080

3. Service Interfaces

This section documents the RESTful service interfaces exposed by each microservice. All endpoints are accessible through the API Gateway at the base URL <http://localhost:8080>. The gateway forwards each request to the correct downstream service after JWT validation.

3.1 Authentication Service (/api/auth)

The authentication service handles all identity-related operations including registration, login, OTP verification, and password management. It issues JWT tokens upon successful authentication, which the gateway validates on all subsequent requests.

Method	Endpoint	Description
POST	/api/auth/send-otp	Send a one-time password to the user's email for verification
POST	/api/auth/verify-otp	Validate the OTP code entered by the user
POST	/api/auth/register	Register a new user account (Patient or Doctor)
POST	/api/auth/login	Authenticate with email and password; returns a signed JWT
POST	/api/auth/google	Authenticate using Google OAuth 2.0
POST	/api/auth/forgot-password	Initiate a password reset and dispatch a reset link
POST	/api/auth/reset-password	Complete the password reset using a valid token
GET	/api/auth/me	Retrieve the profile of the currently authenticated user

3.2 User Service (/api/users)

The user service manages patient and administrator accounts, profile information, profile pictures, and uploaded medical documents. It provides full CRUD operations on user records.

Method	Endpoint	Description
POST	/api/users	Create a new user record
GET	/api/users	Retrieve all registered users
GET	/api/users/{id}	Retrieve a specific user by their unique identifier
PUT	/api/users/{id}	Update a user's profile information
DELETE	/api/users/{id}	Permanently delete a user account
POST	/api/users/{id}/upload-profile-pic	Upload a profile picture for the specified user
GET	/api/users/{id}/profile-image	Retrieve the profile picture of a user
POST	/api/users/{userId}/reports	Upload a medical report PDF for a patient
GET	/api/users/{userId}/reports	Retrieve all medical reports belonging to a patient
GET	/api/users/reports/{reportId}	Download a specific medical report by its ID
DELETE	/api/users/reports/{reportId}	Delete a specific medical report

3.3 Doctor Service (/api/doctors)

The doctor service manages doctor profiles, specialisation details, availability schedules, and verification status. It supports filtering by specialisation for patient-facing search.

Method	Endpoint	Description
POST	/api/doctors	Register a new doctor profile in the system
GET	/api/doctors	Retrieve all doctor profiles
GET	/api/doctors/verified	Retrieve only admin-verified doctors visible to patients
GET	/api/doctors/{id}	Retrieve a doctor by their unique identifier
GET	/api/doctors/email/{email}	Look up a doctor using their email address
PUT	/api/doctors/{id}	Update a doctor's profile details
DELETE	/api/doctors/{id}	Remove a doctor profile from the system
GET	/api/doctors/specialization/{spec}	Filter and retrieve doctors by medical specialisation
POST	/api/doctors/{id}/upload-profile-pic	Upload a profile picture for a doctor
GET	/api/doctors/{id}/profile-image	Retrieve a doctor's profile picture

3.4 Prescription Service (/api/prescriptions)

The prescription service allows doctors to issue digital prescriptions. Prescriptions may include an attached document, and patients can retrieve and download them at any time.

Method	Endpoint	Description
POST	/api/prescriptions	Create a new prescription (supports multipart with document)
GET	/api/prescriptions/{id}	Retrieve a specific prescription by ID
GET	/api/prescriptions/patient/{patientId}	Retrieve all prescriptions issued to a patient
GET	/api/prescriptions/doctor/{doctorId}	Retrieve all prescriptions issued by a doctor
GET	/api/prescriptions/{id}/document	Download the prescription document attachment

3.5 Appointment Service (/api/appointments)

The appointment service manages the full lifecycle of a medical appointment, from initial booking through status updates to cancellation. It supports queries by both patient and doctor.

Method	Endpoint	Description
POST	/api/appointments	Book a new appointment
GET	/api/appointments	Retrieve all appointments in the system
GET	/api/appointments/{id}	Retrieve a specific appointment by ID
GET	/api/appointments/user/{userId}	Retrieve all appointments for a given patient
GET	/api/appointments/doctor/{doctorId}	Retrieve all appointments assigned to a doctor
PUT	/api/appointments/{id}	Update appointment details

Method	Endpoint	Description
PUT	/api/appointments/{id}/status	Update the status of an appointment
DELETE	/api/appointments/{id}	Cancel or delete an appointment

3.6 Payment Service (/api/payments)

The payment service integrates with Stripe Payment Intents to securely process consultation fees. Stripe webhook callbacks are handled to asynchronously update payment statuses.

Method	Endpoint	Description
POST	/api/payments	Initiate a new payment for a consultation
GET	/api/payments/{id}	Retrieve a specific payment record by ID
GET	/api/payments/user/{userId}	Retrieve all payments made by a patient
GET	/api/payments	Retrieve all platform payment records
PUT	/api/payments/{id}/status	Manually update a payment status
POST	/api/payments/notify	Stripe webhook endpoint for asynchronous payment events
GET	/api/payments/admin/stats	Retrieve aggregated payment and revenue statistics

3.7 Notification Service (/api/notifications)

The notification service dispatches real-time communications to users. It uses Twilio for SMS and JavaMail over Gmail SMTP for email alerts, triggered by appointment confirmations, payment receipts, and reminders.

Method	Endpoint	Description
POST	/api/notifications	Create and store a notification record
POST	/api/notifications/send	Immediately dispatch an SMS or email notification
GET	/api/notifications/user/{userId}	Retrieve all notifications for a given user
PUT	/api/notifications/{id}/read	Mark a specific notification as read
DELETE	/api/notifications/{id}	Delete a notification record

3.8 Telemedicine Service (/api/v1/telemedicine/sessions)

The telemedicine service manages video consultation sessions using Jitsi Meet. It generates unique session room URLs tied to appointment records and tracks session start and end times.

Method	Endpoint	Description
POST	/api/v1/telemedicine/sessions	Create a new video consultation session
GET	/api/v1/telemedicine/sessions/{id}	Retrieve a session by its unique identifier
GET	/api/v1/telemedicine/sessions/appointment/{aptId}	Retrieve the session linked to an appointment
PUT	/api/v1/telemedicine/sessions/{id}/start	Mark a session as started

Method	Endpoint	Description
PUT	/api/v1/telemedicine/sessions/{id}/end	Mark a session as ended
GET	/api/v1/telemedicine/sessions/doctor/{doctorId}	Retrieve all sessions for a doctor
GET	/api/v1/telemedicine/sessions/patient/{patientId}	Retrieve all sessions for a patient

3.9 Symptom Checker Service (/api/symptoms)

The symptom checker accepts free-text symptom descriptions from patients, forwards them to the Google Gemini API, and returns a preliminary health analysis with recommended specialisations. All results are persisted.

Method	Endpoint	Description
POST	/api/symptoms/check	Submit symptom data for AI-based analysis via Google Gemini
GET	/api/symptoms/history/{userId}	Retrieve symptom check history for a user
GET	/api/symptoms/{id}	Retrieve a specific symptom check result by ID

3.10 Admin Endpoints (/api/admin)

The admin endpoints provide the platform administrator with controls for user management, doctor verification, appointment oversight, and financial monitoring. These routes are protected by role-based access control and are only accessible to users with the ADMIN role.

Method	Endpoint	Description
GET	/api/admin/users	Retrieve all registered users on the platform
GET	/api/admin/users/stats	Retrieve aggregated user statistics
GET	/api/admin/users/role/{role}	Filter users by their assigned role
DELETE	/api/admin/users/{id}	Permanently delete a user account
POST	/api/admin/users/{id}/suspend	Suspend a user account
POST	/api/admin/users/{id}/unsuspend	Reinstate a suspended user account
PUT	/api/admin/users/{id}/approve	Approve a pending user registration
PUT	/api/admin/users/{id}/reject	Reject a pending user registration
GET	/api/admin/doctors/unverified	Retrieve all doctors awaiting verification
GET	/api/admin/doctors/verified	Retrieve all verified doctors
GET	/api/admin/doctors/stats	Retrieve doctor-related statistics
PUT	/api/admin/doctors/{id}/verify	Approve and verify a doctor registration
PUT	/api/admin/doctors/{id}/reject	Reject a doctor registration
GET	/api/admin/appointments	Retrieve all appointments across the platform
GET	/api/admin/appointments/stats	Retrieve appointment statistics
GET	/api/admin/payments	Retrieve all payment records

Method	Endpoint	Description
GET	/api/admin/payments/stats	Retrieve revenue and payment statistics

4. Key Workflows

This section describes the primary end-to-end workflows within the Clinexa platform. Each workflow represents a sequence of interactions across multiple services, coordinated through the API Gateway.

4.1 Patient Registration and Authentication Workflow

Step 1: The patient submits their email address to `/api/auth/send-otp`.

Step 2: The notification service dispatches a one-time password to the provided email address.

Step 3: The patient submits the OTP to `/api/auth/verify-otp` for validation.

Step 4: Upon successful OTP verification, the patient completes registration via `/api/auth/register`, providing their name, password, and assigned role.

Step 5: The user service creates the account and stores the BCrypt-hashed password in `user_db`.

Step 6: On subsequent visits, the patient authenticates via `/api/auth/login` and receives a signed JWT token.

Step 7: The JWT is included in the Authorization header of all subsequent API requests and is validated by the gateway before each request is forwarded.

4.2 Appointment Booking Workflow

Step 1: The patient browses available doctors by specialisation using `/api/doctors/specialization/{spec}`.

Step 2: The patient selects an available doctor and submits a booking request to `/api/appointments`.

Step 3: The appointment service creates the record in `appointment_db` with an initial status of PENDING.

Step 4: The notification service is triggered to send an SMS and email confirmation to both the patient and the doctor.

Step 5: The doctor reviews the request and accepts or rejects it via `/api/appointments/{id}/status`.

Step 6: If accepted, the patient proceeds to the payment flow before the consultation is confirmed.

4.3 Payment Processing Workflow

Step 1: Upon appointment acceptance, the patient initiates payment through `/api/payments`.

Step 2: The payment service communicates with the Stripe Payment Intents API to create a payment intent and returns a client secret to the frontend.

Step 3: The React frontend renders the Stripe Elements card input, and the patient completes the transaction securely.

Step 4: Stripe asynchronously posts a webhook event to `/api/payments/notify` upon payment completion.

Step 5: The payment service updates the payment status in `payment_db` and triggers a confirmation notification.

Step 6: The appointment status is updated to CONFIRMED.

4.4 Telemedicine Session Workflow

Step 1: At the scheduled appointment time, the system creates a session record via `/api/v1/telemedicine/sessions`.

Step 2: The telemedicine service generates a unique Jitsi Meet room URL tied to the appointment ID.

Step 3: Both the patient and the doctor receive the room link and join the session through the browser-based Jitsi Meet interface.

Step 4: The session is marked as started by calling `/api/v1/telemedicine/sessions/{id}/start`.

Step 5: Following the consultation, the doctor issues a digital prescription through `/api/prescriptions`.

Step 6: The session is formally closed by calling `/api/v1/telemedicine/sessions/{id}/end`, and a post-consultation notification is dispatched.

4.5 Doctor Verification Workflow

Step 1: A new doctor completes registration through the standard auth flow, specifying the DOCTOR role and providing their SLMC registration number.

Step 2: The doctor profile is created with a verification status of UNVERIFIED, making the doctor invisible to patients.

Step 3: The platform administrator reviews the pending registration through `/api/admin/doctors/unverified`.

Step 4: The admin approves the registration using `/api/admin/doctors/{id}/verify` or rejects it with a reason via `/api/admin/doctors/{id}/reject`.

Step 5: Upon approval, the doctor's profile becomes visible in patient-facing search results returned by `/api/doctors/verified`.

Step 6: The doctor receives an email notification confirming their verification status.

4.6 AI Symptom Checking Workflow

Step 1: The patient navigates to the Symptom Checker feature on the platform.

Step 2: The patient enters a natural language description of their symptoms into the provided form.

Step 3: The frontend submits the input to `/api/symptoms/check`.

Step 4: The symptom checker service constructs a structured prompt and forwards it to the Google Gemini API.

Step 5: Gemini returns a preliminary health analysis and a list of recommended medical specialisations.

Step 6: The response is persisted in `symptom_checker_db` and returned to the patient alongside relevant doctor recommendations.

5. Security and Authentication Mechanisms

Security is enforced at multiple layers of the Clinexa platform, from the API gateway down to the service and data tiers.

5.1 JWT-Based Stateless Authentication

All authenticated endpoints are protected using JSON Web Tokens. Upon successful login, the authentication service generates a signed JWT containing the user's ID, email, and assigned role. This token is returned to the client and must be included in the Authorization header of every subsequent request using the Bearer scheme. The Spring Cloud Gateway intercepts each incoming request, extracts the token, and validates the signature and expiry before forwarding the request. Requests with missing, expired, or tampered tokens are rejected with a 401 Unauthorized response. This stateless design eliminates the need for server-side session storage and makes the system horizontally scalable.

5.2 Role-Based Access Control

The platform defines three distinct roles: PATIENT, DOCTOR, and ADMIN. Each role is encoded within the JWT payload and is enforced at both the gateway and individual service levels. Admin-specific routes under `/api/admin` are restricted exclusively to users with the ADMIN role. Doctor-specific operations such as issuing prescriptions and updating availability are protected to ensure only authorised practitioners can perform them. Role validation is implemented using Spring Security annotations such as `@PreAuthorize` on controller methods.

5.3 Password Security

User passwords are never stored in plain text. During registration, passwords are hashed using the BCrypt algorithm with an adaptive cost factor, making them resistant to brute-force and rainbow table attacks. During login, the submitted password is compared against the stored hash using BCrypt's built-in comparison function. No password is ever returned in any API response.

5.4 OTP-Based Email Verification

To prevent unauthorised registrations and ensure email address validity, a one-time password is sent to the user's email before account creation is permitted. The OTP is time-limited and single-use, reducing the risk of account takeover through email enumeration attacks.

5.5 Secrets and Environment Management

All sensitive credentials, including Stripe API keys, Twilio authentication tokens, Google Gemini API keys, and Gmail SMTP credentials, are managed through Kubernetes Secrets and Docker environment variables. These values are injected into containers at runtime and are never embedded in the source code or committed to version control. This follows the Twelve-Factor App principles for configuration management.

5.6 HTTPS and Transport Security

All communications between the frontend and the API gateway are served over HTTPS in a production environment. Stripe's payment flow inherently enforces secure transport, and Jitsi Meet sessions are delivered over encrypted WebRTC channels, ensuring that patient-doctor video consultations remain private and tamper-resistant.

6. Deployment Guide

The Clinexa platform supports two primary deployment modes: Docker Compose for local development and testing, and Kubernetes for production-grade orchestration.

6.1 Prerequisites

- Docker Desktop (v24 or above) with Docker Compose V2
- Kubernetes cluster — Minikube or Kind for local deployment
- kubectl CLI tool configured for the target cluster
- Java 17 JDK (for building backend services locally if required)
- Node.js 18 or above (for frontend development builds)

6.2 Docker Compose Deployment

Docker Compose is the recommended method for local development. All services, databases, and the frontend are defined in the root `docker-compose.yml` file.

```
# Clone the repository
git clone https://github.com/[group-repo]/clinexa.git
cd clinexa

# Copy and configure environment variables
cp .env.example .env
# Edit .env to add Stripe, Twilio, Gemini, Gmail credentials

# Build and start all services
docker-compose up --build -d

# Verify all containers are running
docker-compose ps

# Frontend:      http://localhost:3000
# API Gateway:  http://localhost:8080
```

6.3 Kubernetes Deployment

For orchestrated deployment using Kubernetes, all manifest files are located in the `k8s/` directory of the repository. Each service has a corresponding Deployment, Service, and ConfigMap or Secret resource.

```
# Start Minikube
minikube start

# Apply Kubernetes secrets
kubectl apply -f k8s/secrets/

# Deploy all services
kubectl apply -f k8s/

# Check pod status
kubectl get pods

# Access via port-forward
kubectl port-forward svc/api-gateway 8080:8080
kubectl port-forward svc/frontend 3000:3000
```

6.4 Service Port Reference

Service	Port	Notes
React.js Frontend	3000	Served by Nginx inside Docker container
API Gateway	8080	Single entry point for all API requests
User Service	8081	Connects to user_db PostgreSQL instance
Doctor Service	8082	Connects to doctor_db PostgreSQL instance
Appointment Service	8083	Connects to appointment_db PostgreSQL instance
Notification Service	8084	Connects to Twilio and Gmail SMTP externally
Payment Service	8085	Connects to Stripe API externally
Telemedicine Service	8086	Generates Jitsi Meet room URLs
Symptom Checker Service	8087	Connects to Google Gemini API externally

7. Individual Contributions

The following table outlines the specific responsibilities undertaken by each group member throughout the design, development, and deployment phases of the Clinexa platform.

Member	Registration No.	Contributions
Joel Nithushan A.T (Team Leader)	IT23556652	System architecture design; API Gateway (Spring Cloud Gateway); User Service (authentication, profiles, medical reports); Doctor Service; Admin Dashboard (React.js); Docker Compose configuration; Kubernetes manifests; CI/CD pipeline with multi-stage Docker builds; project coordination.
Vaishnavi L	IT23717336	Appointment Service (booking, scheduling, status management); Appointment scheduling UI (React.js); Prescription module (doctor-side issuance and patient-side viewing).
Thushalini U	IT23794870	Payment Service (Stripe Payment Intents integration); Transactions and payment history UI; Payment report generation and admin financial overview.
Kanistan T	IT23748644	Notification Service (Twilio SMS and JavaMail/Gmail SMTP); Telemedicine Service (Jitsi Meet session management); Symptom Checker Service (Google Gemini AI integration).

8. Conclusion

The Clinexa platform demonstrates that a microservices architecture can be effectively applied to the healthcare domain to produce a scalable, maintainable, and feature-rich telemedicine solution. By decomposing the system into eight independently deployable services, the team was able to work concurrently on separate functional areas while maintaining clear boundaries between components.

The integration of real-world third-party services — Stripe for payments, Twilio and JavaMail for notifications, Jitsi Meet for video consultations, and Google Gemini for AI-powered symptom analysis — demonstrates the practical applicability of the platform beyond an academic exercise. The use of Spring Cloud Gateway as a centralised entry point simplified cross-cutting concerns such as authentication, logging, and routing without duplicating logic across services.

From an infrastructure perspective, containerisation through Docker and orchestration via Kubernetes provides a deployment model that closely mirrors industry practices. The database-per-service pattern ensures data isolation and allows each service to evolve its schema independently. JWT-based stateless authentication and role-based access control provide a security model that is both practical and extensible.

Overall, the Clinexa platform satisfies all requirements outlined in the SE3020 Distributed Systems assignment specification and provides a solid foundation upon which additional features, such as real-time chat, appointment reminders, and enhanced analytics, could be built in future iterations.

Appendix A — Code Listings

The following code excerpts are taken directly from the Clinexa source code. They illustrate the most critical implementation patterns used across the platform. Full source code is available in the GitHub repository linked in submission.txt.

A.1 API Gateway — application.yml (Routing Configuration)

The following configuration defines how the Spring Cloud Gateway routes incoming requests to the appropriate downstream microservice. The JWT authentication filter is applied globally.

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: http://user-service:8081
          predicates:
            - Path=/api/users/**, /api/auth/**
          filters:
            - JwtAuthenticationFilter
        - id: doctor-service
          uri: http://doctor-service:8082
          predicates:
            - Path=/api/doctors/**
        - id: appointment-service
          uri: http://appointment-service:8083
          predicates:
            - Path=/api/appointments/**
        - id: payment-service
          uri: http://payment-service:8085
          predicates:
            - Path=/api/payments/**
        - id: telemedicine-service
          uri: http://telemedicine-service:8086
          predicates:
            - Path=/api/v1/telemedicine/**
        - id: symptom-checker
          uri: http://symptom-checker:8087
          predicates:
            - Path=/api/symptoms/**
```

A.2 User Service — JwtUtil.java (Token Generation and Validation)

This utility class is responsible for generating signed JWT tokens on login and validating them on each incoming request at the gateway level.

```
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secretKey;

    private static final long EXPIRATION_MS = 86400000; // 24 hours

    public String generateToken(String email, String role, Long userId) {
        return Jwts.builder()
            .setSubject(email)
            .claim("role", role)
            .claim("userId", userId)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
```

```
        .signWith(SignatureAlgorithm.HS256, secretKey)
        .compact();
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
            .setSigningKey(secretKey)
            .parseClaimsJws(token)
            .getBody();
    }

    public boolean isTokenValid(String token) {
        try {
            extractAllClaims(token);
            return !isTokenExpired(token);
        } catch (JwtException e) {
            return false;
        }
    }

    private boolean isTokenExpired(String token) {
        return extractAllClaims(token).getExpiration().before(new Date());
    }
}
```

A.3 Appointment Service — AppointmentController.java

This controller handles the core appointment booking and management operations, demonstrating RESTful endpoint design with proper HTTP method usage.

```
@RestController
@RequestMapping("/api/appointments")
@RequiredArgsConstructor
public class AppointmentController {

    private final AppointmentService appointmentService;

    @PostMapping
    public ResponseEntity<AppointmentDTO> createAppointment(
        @RequestBody @Valid AppointmentRequest request) {
        AppointmentDTO created = appointmentService.createAppointment(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @GetMapping("/user/{userId}")
    public ResponseEntity<List<AppointmentDTO>> getByPatient(
        @PathVariable Long userId) {
        return ResponseEntity.ok(appointmentService.getByPatient(userId));
    }

    @PutMapping("/{id}/status")
    public ResponseEntity<AppointmentDTO> updateStatus(
        @PathVariable Long id,
        @RequestBody StatusUpdateRequest request) {
        return ResponseEntity.ok(
            appointmentService.updateStatus(id, request.getStatus()));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> cancelAppointment(@PathVariable Long id) {
        appointmentService.cancelAppointment(id);
        return ResponseEntity.noContent().build();
    }
}
```

A.4 Payment Service — PaymentController.java (Stripe Integration)

This controller demonstrates the Stripe Payment Intents API integration, including webhook handling for asynchronous payment confirmation.

```
@RestController
@RequestMapping("/api/payments")
@RequiredArgsConstructor
public class PaymentController {

    private final PaymentService paymentService;

    @PostMapping
    public ResponseEntity<PaymentResponseDTO> initiatePayment(
        @RequestBody PaymentRequest request) {
        PaymentResponseDTO response = paymentService.createPaymentIntent(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/notify")
    public ResponseEntity<String> handleStripeWebhook(
        @RequestBody String payload,
        @RequestHeader("Stripe-Signature") String sigHeader) {
        paymentService.processWebhookEvent(payload, sigHeader);
        return ResponseEntity.ok("Webhook processed");
    }

    @GetMapping("/admin/stats")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<PaymentStatsDTO> getStats() {
        return ResponseEntity.ok(paymentService.getStatistics());
    }
}
```

A.5 Symptom Checker — SymptomCheckController.java (Google Gemini AI)

This controller demonstrates the Google Gemini AI integration. Patient-reported symptoms are forwarded to Gemini, and the structured response is persisted and returned to the frontend.

```
@RestController
@RequestMapping("/api/symptoms")
@RequiredArgsConstructor
public class SymptomCheckController {

    private final GeminiService geminiService;
    private final SymptomCheckRepository repository;

    @PostMapping("/check")
    public ResponseEntity<SymptomResultDTO> checkSymptoms(
        @RequestBody SymptomRequest request) {

        String prompt = "You are a medical assistant. A patient reports: "
            + request.getSymptoms()
            + ". Provide: (1) a brief preliminary analysis, "
            + "(2) possible conditions, (3) recommended specialisations. "
            + "Remind the patient to consult a qualified doctor.";

        String geminiResponse = geminiService.query(prompt);

        SymptomCheck record = SymptomCheck.builder()
            .userId(request.getUserId())
```

```
        .symptoms(request.getSymptoms())
        .aiResponse(geminiResponse)
        .checkedAt(LocalDate.now())
        .build();

    repository.save(record);
    return ResponseEntity.ok(new SymptomResultDTO(record.getId(),
geminiResponse));
    }
}
```

A.6 Docker Compose Configuration (Excerpt)

The following excerpt from docker-compose.yml illustrates the container definitions for the API gateway, user service, payment service, and frontend.

```
version: "3.8"
services:

  api-gateway:
    build: ./api-gateway
    ports:
      - "8080:8080"
    environment:
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      - user-service
      - doctor-service

  user-service:
    build: ./user-service
    ports:
      - "8081:8081"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:postgresql://user-db:5432/user_db
      - SPRING_DATASOURCE_USERNAME=${DB_USER}
      - SPRING_DATASOURCE_PASSWORD=${DB_PASSWORD}
    depends_on:
      - user-db

  user-db:
    image: postgres:16
    environment:
      - POSTGRES_DB=user_db
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - user-db-data:/var/lib/postgresql/data

  payment-service:
    build: ./payment-service
    ports:
      - "8085:8085"
    environment:
      - STRIPE_SECRET_KEY=${STRIPE_SECRET_KEY}
      - STRIPE_WEBHOOK_SECRET=${STRIPE_WEBHOOK_SECRET}

  symptom-checker:
    build: ./symptom-checker-service
    ports:
      - "8087:8087"
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}

  frontend:
```

```
    build: ./healthcare-frontend
    ports:
      - "3000:80"

volumes:
  user-db-data:
  doctor-db-data:
  appointment-db-data:
  payment-db-data:
  notification-db-data:
  telemedicine-db-data:
  symptom-checker-db-data:
```

End of Report — Clinexa Healthcare Platform | Group NU_04 | SE3020 Distributed Systems | SLIIT 2026